Homework 07

1(a)

Class	Observations	Estimated prior
0	302	0.65368
1	160	0.34632

Predictor	Class 0 mean	Class 1 mean
sbp	135.4603	143.7375
tobacco	2.6347	5.5249
ldl	4.3442	5.4879
adiposity	23.9691	28.1202
famhist	1.3179	1.6000
typea	52.3675	54.4937
obesity	25.7375	26.6229
alcohol	15.9314	19.1452
age	38.8543	50.2938

Pooled covariance denominator: **n - K = 462 - 2 = 460**.

Pooled covariance matrix

Values below are rounded to two decimals; predictions use full precision.

	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age
sbp	405.43	14.59	4.58	49.16	0.34	-15.59	18.94	64.40	95.13
tobacco	14.59	19.24	0.76	7.54	0.02	-2.06	1.83	20.52	22.77
ldl	4.58	0.76	4.00	6.03	0.09	0.34	2.66	-2.53	6.48
adiposity	49.16	7.54	6.03	56.75	0.43	-5.31	22.71	16.12	60.51
famhist	0.34	0.02	0.09	0.43	0.23	0.08	0.18	0.77	1.00
typea	-15.59	-2.06	0.34	-5.31	0.08	95.57	2.64	7.96	-20.28
obesity	18.94	1.83	2.66	22.71	0.18	2.64	17.62	4.69	15.70
alcohol	64.40	20.52	-2.53	16.12	0.77	7.96	4.69	598.28	27.89
age	95.13	22.77	6.48	60.51	1.00	-20.28	15.70	27.89	184.13

1(b)-(c)

(b)

Dimensions: X is 462 x 9; y has 462 entries; the mean matrix is 2 x 9; the pooled covariance is 9 x 9; the weight matrix is 9 x 2; and the score matrix is 462 x 2.

Observed / Predicted	0	1
0	258	44
1	73	87

(c)

Manual LDA and scikit-learn LDA (solver="svd", empirical priors) give this same confusion matrix and match on all 462 labels. The in-sample error is (44 + 73) / 462 = **25.32%**.

Python code

```
import numpy as np
import pandas as pd
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
heart = pd.read_csv("saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})
X = heart.drop(columns="chd").to_numpy(dtype=float)
y = heart["chd"].to_numpy()
classes, counts = np.unique(y, return_counts=True)
n, p = X.shape
priors = counts / n
means = np.vstack([X[y == k].mean(axis=0) for k in classes])
within_ss = np.zeros((p, p))
for j, k in enumerate(classes):
    centered = X[y == k] - means[j]
    within_ss += centered.T @ centered
pooled_cov = within_ss / (n - len(classes))
weights = np.linalg.solve(pooled_cov, means.T)
intercepts = -0.5 * np.sum(means.T * weights, axis=0) + np.log(priors)
scores = X @ weights + intercepts
manual = classes[scores.argmax(axis=1)]
lda = LinearDiscriminantAnalysis(solver="svd", priors=priors).fit(X, y)
assert np.array_equal(manual, lda.predict(X))
print(priors, means, pooled_cov, sep="\n")
print(confusion_matrix(y, manual, labels=classes))
print(confusion_matrix(y, lda.predict(X), labels=classes))
```

2(a), (c)

Confusion matrices: **observed digits in rows, predicted digits in columns**.

(a) 40 predictors: 312 / 2,007 errors = 15.55%

Obs. / Pred.	0	1	2	3	4	5	6	7	8	9
0	319	0	24	6	2	2	1	0	5	0
1	0	238	5	0	11	0	7	0	3	0
2	8	0	168	8	8	3	0	0	3	0
3	5	0	4	128	0	14	0	0	15	0
4	0	2	13	1	150	1	0	1	7	25
5	12	0	3	5	1	131	0	0	6	2
6	0	0	8	0	1	8	148	0	5	0
7	1	0	6	1	6	0	0	124	2	7
8	1	0	10	12	3	2	0	0	137	1
9	0	0	1	2	7	1	1	4	9	152

(c) 10 predictors: 687 / 2,007 errors = 34.23%

Obs. / Pred.	0	1	2	3	4	5	6	7	8	9
0	298	0	11	1	1	16	28	4	0	0
1	0	241	10	0	3	1	1	0	2	6
2	19	2	95	14	14	3	22	21	5	3
3	5	0	6	103	3	42	0	0	7	0
4	1	5	14	0	95	2	4	15	17	47
5	15	0	6	25	4	96	4	2	8	0
6	41	0	15	4	0	18	78	7	5	2
7	4	1	6	0	6	0	1	122	0	7
8	1	2	11	36	12	11	4	1	74	14
9	0	2	9	2	20	0	0	16	10	118

Reducing the model from 40 to 10 predictors added **375 test errors**, an increase of **18.68 percentage points**. Both versions fitted successfully.

2(b)-(c)

(b)

QDA worked with 40 predictors: every within-class covariance matrix had rank 40, and test accuracy was 84.45%. Reducing the dimension makes covariance estimation more manageable and can remove constant or redundant pixels.

(c)

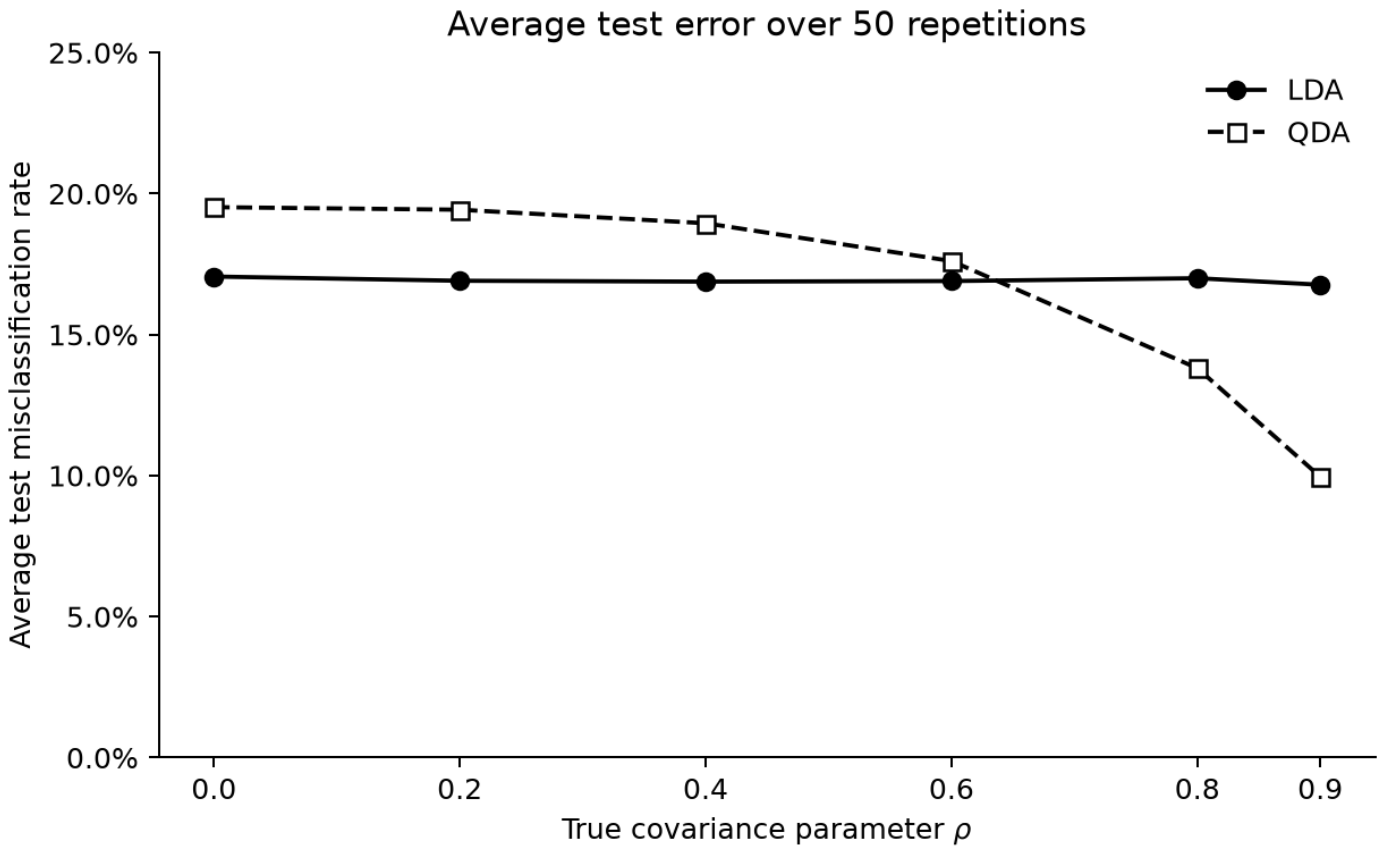
The 10-predictor model also had full-rank covariance matrices. It estimates 55 covariance parameters per class instead of 820, which can reduce estimation variance. However, the higher test error suggests that losing information about digit shape outweighed this benefit.

```
import numpy as np
import pandas as pd
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
train = pd.read_csv("zip-train.csv.gz", header=None)
test = pd.read_csv("zip-test.csv.gz", header=None)
X_train = train.iloc[:, 1:].to_numpy(dtype=float)
y_train = train.iloc[:, 0].to_numpy(dtype=int)
X_test = test.iloc[:, 1:].to_numpy(dtype=float)
y_test = test.iloc[:, 0].to_numpy(dtype=int)
# Screening uses training data only, without standardization.
order = np.argsort(-X_train.var(axis=0, ddof=1), kind="stable")
for m in (40, 10):
    selected = order[:m]
    A, B = X_train[:, selected], X_test[:, selected]
    model = QuadraticDiscriminantAnalysis(tol=1e-8)
    model.fit(A, y_train)
    predicted = model.predict(B)
    ranks = [np.linalg.matrix_rank(np.cov(A[y_train == k], rowvar=False))
             for k in range(10)]
    print("Predictors:", m, "Class covariance ranks:", ranks)
    print(confusion_matrix(y_test, predicted, labels=np.arange(10)))
    print("Errors:", np.sum(predicted != y_test))
    print("Test error rate:", np.mean(predicted != y_test))
```

3(a)-(b)

Seed: 432. Test error rates for one simulation and averages over 50 fresh repetitions:

rho	Single LDA	Single QDA	Mean LDA	Mean QDA
0	0.1710	0.2010	0.1706	0.1952
0.2	0.1715	0.1815	0.1691	0.1943
0.4	0.1575	0.1715	0.1688	0.1895
0.6	0.1535	0.1610	0.1690	0.1761
0.8	0.1815	0.1495	0.1700	0.1381
0.9	0.1815	0.1030	0.1677	0.0996



3(c)

At rho = 0, the classes have the same covariance matrix, so LDA's assumption is correct. QDA estimates two matrices unnecessarily and has greater estimation variance. With ten predictors, LDA estimates 55 covariance parameters and QDA estimates 110 across the two classes. LDA had lower average error through rho = 0.6.

At larger rho, the class correlations differ more strongly. QDA can use this difference through a quadratic boundary, reducing model bias enough to offset its extra variance. At rho = 0.9, its average error was 9.96%, versus 16.77% for LDA. The opposite correlations cancel in the population pooled covariance, so the population LDA rule depends on the sign of X1. Its marginal distributions do not change with rho, explaining LDA's nearly flat curve.

The eight noise predictors have no class information, but their fitted means and covariances fluctuate with the training sample. Those fluctuations can distort predictions, especially for QDA because it estimates more parameters separately in each class.

```
import numpy as np
from sklearn.discriminant_analysis import (LinearDiscriminantAnalysis,
                                           QuadraticDiscriminantAnalysis)
rng = np.random.default_rng(432)
rhos = [0, .2, .4, .6, .8, .9]
def sample(rho, n):
    blocks = []
    for mean, corr in [(-1, rho), (1, -rho)]:
        a = rng.multivariate_normal([mean, 0], [[1, corr], [corr, 1]], n)
        blocks.append(np.column_stack([a, rng.standard_normal((n, 8))]))
    return np.vstack(blocks), np.repeat([1, 2], n)
def compare(rho):
    X, y = sample(rho, 100)
    T, target = sample(rho, 1000)
    models = [LinearDiscriminantAnalysis(priors=[.5, .5]),
              QuadraticDiscriminantAnalysis(priors=[.5, .5])]
    return [np.mean(m.fit(X, y).predict(T) != target) for m in models]
single = np.array([compare(rho) for rho in rhos])
means = np.array([np.mean([compare(rho) for _ in range(50)], axis=0)
                  for rho in rhos])
print(single, means, sep="\n")
import matplotlib.pyplot as plt
plt.plot(rhos, means[:, 0], "o-", label="LDA")
plt.plot(rhos, means[:, 1], "s-", label="QDA")
plt.xlabel("True covariance parameter rho")
plt.ylabel("Average test misclassification rate")
plt.legend(); plt.tight_layout(); plt.show()
```

Predicting coronary heart disease

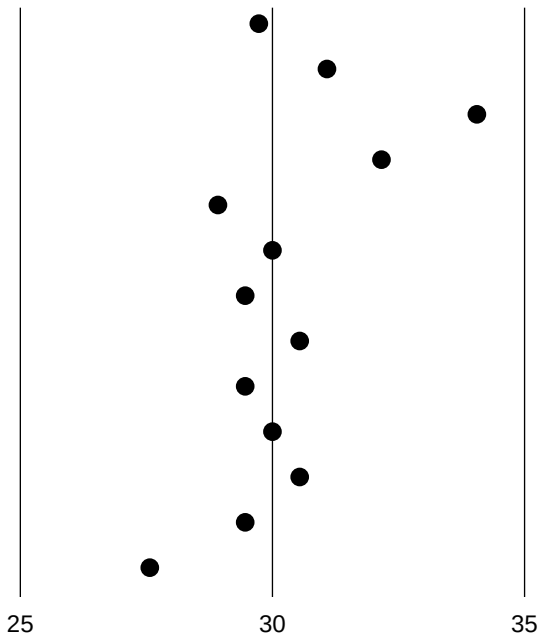
STAT 432 / Homework 7, Question 4 / R + MASS / Seed 432

Data. I used the 462 observations in SAheart to predict **chd**; 160 people (34.6%) had CHD. The nine predictors were sbp, tobacco, ldl, adiposity, famhist, typea, obesity, alcohol and age. I coded family history as 1 for Absent and 2 for Present. There were no missing values, nonfinite values or duplicate rows, so I kept all observations.

Method. I first set aside 92 observations for testing, preserving the class proportions. I compared models using the same five stratified cross-validation (CV) folds in the remaining 370 observations. Imputation, scaling, PCA and class priors were fitted only on each fold's training portion. I selected the model with the fewest total CV errors, then refitted it on all 370 training observations.

TRAINING-ONLY COMPARISON

Candidate	Errors / 370	CV error
LDA	110	29.73%
QDA	115	31.08%
PCA 2 + LDA	126	34.05%
PCA 2 + QDA	119	32.16%
PCA 4 + LDA	107	28.92%
PCA 4 + QDA	111	30.00%
PCA 6 + LDA	109	29.46%
PCA 6 + QDA	113	30.54%
PCA 8 + LDA	109	29.46%
PCA 8 + QDA	111	30.00%
Blend (w = 0.25)	113	30.54%
Blend (w = 0.5)	109	29.46%
Blend (w = 0.75)	**102**	**27.57%**



PCA models use the stated number of components after standardizing the predictors. Blends combine class probabilities as *w* P(LDA) + (1 - *w*) P(QDA). All 13 models fitted successfully, with no warnings or singular covariance matrices.

SELECTED MODEL & TEST RESULT

I selected **75% LDA + 25% QDA** because it had the lowest CV error: **102/370 (27.57%)**, compared with 29.73% for LDA and 31.08% for QDA. I then evaluated this model once on the reserved test set. It misclassified **19 of 92 observations (20.65%)**. The test results were not used to choose or change the model.

INTERPRETATION & LIMITATIONS

Adding some QDA weight helped in CV, although the improvement over LDA alone was small. QDA can work better when class covariances differ, but estimating more parameters can make it less stable with small samples or strongly related predictors. No singular covariance matrices occurred here. Trying 13 models may make the winning CV error look better than it will on new data. The test set contained only 92 observations, so its error rate could change noticeably with a different split. Also, overall error does not show how well the model identifies CHD cases specifically.

Appendix A - Complete SKILL.md

The complete skill I used is reproduced below.

```
---
name: lda-qda-analysis-r
description: Explicit R LDA/QDA analysis with a one-page statistical report.
---

# Invocation and scope

Apply only when explicitly requested. Require the dataset, response variable, predictors, and
any necessary coding instructions; ask for missing inputs.

Use R with MASS::lda and MASS::qda for binary/multiclass classification. Allow only LDA, QDA,
or their combinations. Make no dataset-specific assumptions. Explain when data cannot support
valid analysis.

# Analysis workflow

1. Inspect schema, classes, predictor types/coding, missing/nonfinite values, duplicates,
identifiers, leakage, imbalance, and group/time dependence. Document exclusions; never impute
responses. Before splitting, perform only integrity checks, not outcome-driven selection.

2. Set/report an integer seed with set.seed(). Reserve a stratified 20% test set before
preprocessing, selection, or tuning; honor supplied split rules and group/time dependence.
Save split indices. If a valid split/CV is impossible, explain and obtain a defensible
design. Lock the test set until selection is final.

3. Create shared stratified 5-fold training-only CV, reducing folds if needed for class
counts. Respect group/time constraints. Save and reuse identical folds for all
candidates/settings; each analysis fold must contain every class.

4. Fit preprocessing separately inside each CV analysis fold, then apply it unchanged to that
fold's validation observations. This includes numeric median/categorical mode imputation,
categorical encoding, zero-variance or dependent-column handling, scaling, feature screening,
and PCA when used. Learn factor levels/novel-level handling within folds; avoid redundant
dummy columns and respect supplied coding. Never preprocess all training data before CV.

5. Include ordinary MASS::lda and MASS::qda as baselines. Use empirical class priors
estimated within each analysis fold unless the user specifies priors. Optionally compare PCA
followed by LDA/QDA using stats::prcomp, or blends w * P_LDA + (1-w) * P_QDA from
predict(...)$posterior, with 0 <= w <= 1. Align posterior columns by class and predict the
maximum probability. Define a compact candidate grid using training data only; tune PCA
dimension and blend weight by CV. Use feasible PCA dimensions in every fold. Document
revisions using training/CV evidence only.

6. Record errors, warnings, covariance rank/conditioning, and class counts. QDA needs enough
observations per class and nonsingular within-class covariance; having more observations than
predictors alone does not guarantee stability. Use tryCatch and explicit warning checks. Mark
failed-fold/invalid-prediction candidates ineligible; report failures, never average only
successful folds. Do not silently regularize QDA. If all candidates fail, explain and stop.

7. Compare pooled out-of-fold misclassification rates: total incorrect predictions divided by
total validation observations. Select the valid model/settings with the lowest rate; break
exact ties by fewer retained dimensions, then LDA before QDA before blends. Record the
selection before opening the test set. Refit its preprocessing and classifier(s) on all
```

training observations. If refitting fails, resolve using training data only and document the change. Evaluate the final selected pipeline exactly once on the reserved test set. Do not evaluate competing models there or retune afterward. Save predictions for report revisions without reevaluation; retain R code, seed, indices, settings, and sessionInfo().

One-page report

Render a readable, professional one-page PDF report containing:

- Dataset purpose, sample sizes, response/classes, predictors, coding, and data issues.
- Candidate models, preprocessing, split, CV folds, selection metric, and seed.
- A compact numerical table of CV misclassification rates and settings for all candidates tested; label failures explicitly. Keep the grid reportable.
- A meaningful labeled figure comparing CV errors, with legible axes and model labels.
- Selected model/settings, selection rationale, and reserved test error as both mistakes/total and a rate.
- Brief interpretation and limitations: QDA can benefit from different class covariances but estimates more parameters and can be unstable with small classes, collinearity, or high dimension. Explain the observed bias-variance trade-off; acknowledge selection optimism in CV and test uncertainty.

Use concise prose, a clean table and compact figure; omit code. Use at least 10-point body/table text and 9-point plot labels. Render, verify the page count, and inspect the page for clipping and readability. Shorten prose or simplify layout instead of shrinking text excessively. Do not claim one-page compliance without verification. Keep code/diagnostics separate.

Appendix B - Explicit invocation and reproducibility

How I invoked the skill

My skill is 4,814 bytes. I saved it in homework/lda-qda-analysis-r/SKILL.md, outside Codex's automatically loaded skill folders, so it would only be used when I explicitly requested it.

I invoked the skill with the prompt below. I reviewed the cross-validation results and did not revise the skill or change the selected model after seeing the test results.

Exact invocation prompt

Explicitly invoke the LDA/QDA analysis skill you just created to complete the analysis portion of Question 4 of STAT 432 Homework 7.

Use `data/saheart.csv`, with `chd` as the response and all nine other variables as numerical predictors. Code `famhist` as 1 for Absent and 2 for Present.

Follow every instruction in SKILL.md, including reserving a test set before model selection, comparing ordinary LDA and QDA, using identical cross-validation folds, preventing data leakage, and evaluating the final model only once on the reserved test set.

Seek the best predictive performance among the methods allowed by the skill, using cross-validation misclassification rate to select the model. Do not choose models or make revisions based on test-set results.

Use R, set and report a random seed, and generate reproducible code.

Produce:

1. A polished, one-page statistical report containing a figure, a table, numerical results, the selected model, test error, and limitations.
2. The R code used to produce the analysis.
3. The complete contents of SKILL.md.
4. A short explanation of how the skill was explicitly invoked, including this invocation prompt and the skill's exact size in bytes.

Ensure the one-page limit applies only to the statistical report, not the appended skill or explanation.

Before finalizing, verify that the reported numerical values match the actual computations and that the figure and table are readable.

Reproducibility

I used R 4.6.1 and MASS 7.3-65. I ran these commands from homework/question-04:

Rscript analysis.R train ../saheart.csv results
Rscript analysis.R test ../saheart.csv results

I saved the split, CV folds, predictions and diagnostics with my R code. Report edits used these saved results without reevaluating the model.